

Examen réparti 1

9 novembre 2021

Antoine Miné

Déroulement de l'examen

L'examen dure 2h.

Tous les documents papier sont autorisés (impressions des transparents, notes personnelles, livres, etc.). L'utilisation de moyens de communication (téléphone, tablette, etc.) est interdite. L'utilisation d'une clé USB est autorisée, en lecture seule. Seul le format FAT est supporté (pas NTFS).

Vous trouverez sur votre compte dans le répertoire de travail `eclipse-workspace` un projet Eclipse `ILP2` contenant les sources d'ILP2. Vous complèterez ce projet en répondant aux questions de l'énoncé. En cas de problème, une archive ZIP des sources d'ILP2 est également disponible sur le site de l'UE : <http://www-master.ufr-info-p6.jussieu.fr/site-annuel-courant/DLP>.

Tous vos fichiers doivent être rédigés dans le package `com.paracamplus.ilp2.partiel2021` et ses sous-packages du projet Eclipse `ILP2`. Ils se trouveront donc dans un sous-répertoire de `eclipse-workspace/ILP2/Java/src/com/paracamplus/ilp2/partiel2021` de votre compte. Placez bien les fichiers demandés dans le package (et donc dans le répertoire) précisé pour chaque question.

En fin d'examen, sauvegardez tous vos fichiers, fermez Eclipse et déloguez-vous. Les fichiers de votre compte seront automatiquement ramassés. L'examen sera corrigé à la main. Quand du code est demandé, les erreurs de syntaxe ne seront pas pénalisées. Il est souhaitable de n'expliciter que les parties du code qui vous semblent importantes, et de remplacer les parties peu informatives par un commentaire indiquant succinctement ce qui a été omis (inutile, par exemple, le détailler du code répétitif dans son intégralité).

Mise en route

- Ouvrez un navigateur et allez à l'URL : <http://www-master.ufr-info-p6.jussieu.fr/site-annuel-courant/DLP>. Vous y trouverez notamment les transparents du cours. Une copie locale de la documentation Java est disponible sur <https://www-ppti.ufr-info-p6.jussieu.fr>.
- Lancez Eclipse en tapant dans un terminal : `eclipse-2020 &`
- Ouvrez le projet `ILP2` contenant les sources d'ILP2 et vérifiez qu'elles compilent bien. Si le projet `ILP2` n'apparaît pas dans le « Package Explorer », ouvrez le menu « File / Open Projects From Filesystem... », cliquez sur « Directory... », choisissez le répertoire `eclipse-workspace/ILP2`, puis « Finish ».
- Si le plug-in ANTLR ne compile pas automatiquement les fichiers de grammaires `.g4` en sources `.java`, vous pouvez taper dans un terminal les commandes suivantes :

```
cd ~/eclipse-workspace/ILP2
./compile_ANTLR.sh
```

Dans Eclipse, cliquez sur le projet dans le « Package explorer » et tapez F5 (ou « Refresh »). Les sources générées par ANTLR 4 devraient apparaître dans `targets/generated-sources/antlr4/antlr4`. Il faut attendre quelques secondes avant qu'elles soient compilées par Eclipse.

Après chaque modification d'un fichier `.g4`, vous devrez à nouveau lancer la commande `./compileANTLR.sh` depuis le répertoire `eclipse-workspace/ILP2`.

- Une cause possible d'erreur de compilation est un environnement Java mal configuré. Faites un clic droit sur le nom du projet dans le « Package Explorer », cliquez sur « Properties », puis vérifiez les éléments suivants :
 - La version de la bibliothèque système (onglet « Java build path / Libraries » puis « JRE System Library ») doit être 8 ou supérieure. Vous pouvez la changer avec le bouton « Edit... / Alternate JRE ». Les JDK disponibles localement sont dans le répertoire `/usr/lib/jvm`.
 - JUnit 4 doit être présent. Sinon, ajoutez-le avec le bouton « Add Library ».
 - Le niveau de compatibilité Java dans l'onglet « Java Compiler » des propriétés doit correspondre à la version du JDK utilisée.
 - La liste des répertoires source doit contenir le répertoire `target/generated-sources/antlr4` (dans « Java build path », onglet « Source »).

Variables entières bornées

Dans cet examen, nous allons ajouter à ILP2 une nouvelle déclaration de variables locales qui contraint la valeur de la variable entre deux bornes entières fixées à la déclaration.

La nouvelle instruction `letrange var : l .. h = expr1 in expr2` est similaire à l'instruction `let var = expr1 in expr2` d'ILP2. Elle introduit dans `expr2` une nouvelle variable `var` initialisée avec la valeur de `expr1`. Cependant, à la différence de `let`, l'instruction `letrange` force la variable `var` à contenir une valeur entière comprise entre `l` et `h`. Ainsi, quand on essaye de stocker une valeur `v` dans `var` :

- si `v` est un entier entre `l` et `h` (au sens large), `var` prend la valeur `v` ;
- si `v` est un entier strictement inférieur à `l`, `var` prend la valeur `l` ;
- si `v` est un entier strictement supérieur à `h`, `var` prend la valeur `h` ;
- si `v` n'est pas un entier, une erreur est signalée.

Par exemple le programme suivant :

```
1 letrange x : 0 .. 9 = 5
2     and y : 0 .. 9 = 12
3     and z : 0 .. 9 = -1 in
4 (print(x); print(y); print (z))
```

affichera : « 590 ». Cet exemple montre qu'il est possible d'introduire plusieurs variables avec bornes simultanément, avec le mot-clé `and`, sur le modèle de `let`.

Le programme suivant :

```
1 letrange t : 0 .. 10 = "help" in print(t)
```

signalera par contre une erreur puisque la valeur d'initialisation n'est pas entière.

Dans `letrange var : l .. h = expr1 in expr2`, la contrainte que `var` est un entier entre `l` et `h` n'est pas uniquement vérifiée à l'initialisation avec la valeur de `expr1`, elle doit également être vérifiée après chaque modification de `var` dans `expr2`. Par exemple :

```
1 letrange x : 1 .. 3 = 1 in
2 let i = x in
3 while i <= 5 do (
4   print(x);
5   i = i + 1;
6   x = x + 1
7 )
```

affichera : « 12333 », puisque `x` ne peut jamais dépasser la valeur 3. Il sera donc nécessaire de se souvenir des valeurs minimales et maximales autorisées pour toute variable bornée `var` durant toute sa durée de vie, et d'effectuer une vérification à chaque affectation dans `var`.

Notons que, dans `letrange var : l .. h = expr1 in expr2`, `l` et `h` ne sont pas forcément des constantes littérales ; ce sont des expressions arbitraires qui doivent être évaluées pour déterminer la valeur maximale et minimale de `var`, comme dans :

```
1 let x = 1 in
2 letrange i : x-1 .. x+1 = x in ...
```

Ces expressions sont évaluées une fois, à la création de `var`, en même temps que `expr1`. C'est une erreur si une d'entre elles s'évalue en une valeur non entière.

Il n'est pas possible de spécifier une valeur minimale et maximale pour une variable globale ou un argument de fonction : notre construction ne concerne que les variables locales.

1 Grammaire et analyse syntaxique

Grammaire (2 pts). Écrivez une grammaire qui étend ILP2 avec la construction :

```
letrange var1 : l1 .. h1 = expr1 and ... and varN : lN .. hN = exprN in expr
```

Cette grammaire se trouvera dans le fichier `eclipse-workspace/ILP2/Java/src/com/paracampus/ilp2/partiel2021/grammar/ILPgrammarPartiel2021.g4`. N'oubliez pas de lancer `./compileANTLR.sh` pour compiler la grammaire en Java.

Arbre syntaxique (2 pts). Proposez une extension de l'arbre syntaxique pour gérer les variables locales avec bornes. Vous fournirez toutes les interfaces correspondantes dans le package `com.paracampus.ilp2.partiel2021.interfaces` et toutes les classes dans le package `com.paracampus.ilp2.partiel2021.ast` sans oublier celles liées aux motifs fabrique (*factory*) et visiteur (*visitor*).

Analyse syntaxique (2 pts). Implantez dans le package `com.paracampus.ilp2.partiel2021.parser` les classes `ILPMLparser` et `ILPMLlistener` implantant l'analyseur syntaxique pour le langage étendu.

2 Interprétation (5 pts)

Implantez dans `com.paracampus.ilp2.partiel2021.interpreter` la classe `Interpreter` qui enrichit l'interprète d'ILP2 avec la construction `letrange`.

Il sera nécessaire de maintenir la valeur minimale et maximale autorisée pour toute variable bornée introduite par la construction `letrange`. Une solution possible est d'enrichir l'environnement lexical pour représenter cette information, mais ce n'est pas la seule solution et vous restez libres d'en choisir une autre. Quelle que soit la solution retenue, s'il est nécessaire d'enrichir les structures de données de l'interprète, vous fournirez le code des classes et interfaces correspondantes dans les packages `com.paracampus.ilp2.partiel2021.interpreter` et `com.paracampus.ilp2.partiel2021.interpreter.interfaces`.

Nous rappelons que les entiers sont représentés dans l'interprète par des objets de classe `BigInteger`. Vous pourrez utiliser les méthodes `compareTo(Object o)` (compare `this` à `o`), `min(BigInteger o)` (retourne le minimum entre `this` et `o`) et `max(BigInteger)` (retourne le maximum entre `this` et `o`).

3 Tests (2 pts)

Proposez une base de tests dans le répertoire `eclipse-workspace/ILP2/Java/src/com/paracampus/ilp2/partiel2021/samples/`.

Vous fournirez au moins 4 tests, *qui ne reprennent pas les exemples de l'énoncé*, en prenant un soin particulier à couvrir l'ensemble des comportements possibles (y compris les cas d'erreur).

Pour chaque test, fournissez le source ILPML mais aussi les fichiers permettant de vérifier les résultats attendus. Vous fournirez également un fichier texte `eclipse-workspace/ILP2/Java/src/com/paracampus/ilp2/partiel2021/samples/tests.txt` précisant, pour chaque test, le comportement ou l'erreur testé.

4 Compilation

Classes et interfaces (2 pt). Donnez la liste des classes et interfaces que vous auriez à ajouter pour implanter le compilateur, en indiquant leur nom, leur package, les interfaces qu'elles implémentent et les classes et interfaces dont elles héritent. On ne demande pas dans cette question le code Java de ces classes et interfaces.

Vous fournirez votre réponse dans un fichier texte : `eclipse-workspace/ILP2/Java/src/com/paracamplus/ilp2/partiel2021/compiler/classes.txt`

Analyse statique (2 pts). Donnez dans la classe `GlobalVariableCollector` du package `com.paracamplus.ilp2.partiel2021.compiler` l'analyse statique des variables globales.

Schéma de compilation (5 pts). Donnez un schéma de compilation détaillé pour :

- la construction `letrange` ;
- l'affectation dans les variables avec bornes.

Vous fournirez votre réponse dans un fichier texte : `eclipse-workspace/ILP2/Java/src/com/paracamplus/ilp2/partiel2021/compiler/schema.txt`. Le code de la classe `Compiler` n'est pas demandé.

Comme pour l'interprète, il sera nécessaire de maintenir la valeur minimale et maximale autorisée pour toute variable bornée. Une solution est d'associer à chaque variable avec borne `var`, une variable `var_l` et `var_h` dans le code C généré pour maintenir cette information. Toutefois, vous êtes libres de choisir une autre solution. Dans tous les cas, si vous êtes amenés à ajouter des classes et interfaces au compilateur pour faciliter la génération de code, vous les listerez dans le fichier `schema.txt` en indiquant leur rôle (le code Java n'est pas demandé). Précisez également dans le fichier `schema.txt` si vous devez apporter ou non des modifications à la bibliothèque d'exécution `ilp.c` et `ilp.h` et, si c'est le cas, décrivez ces modifications (listez en particulier les fonctions C que vous auriez à implanter, en précisant leur rôle et leur fonctionnement, ainsi que les ajouts éventuels à `ILP_Object`).